

AI ASSISTED CODING

Assignment-3.2

B.Tejaswi

2303A52123

Batch-40

Task Description-1

- Progressive Prompting for Calculator Design: Ask the AI to design a simple calculator

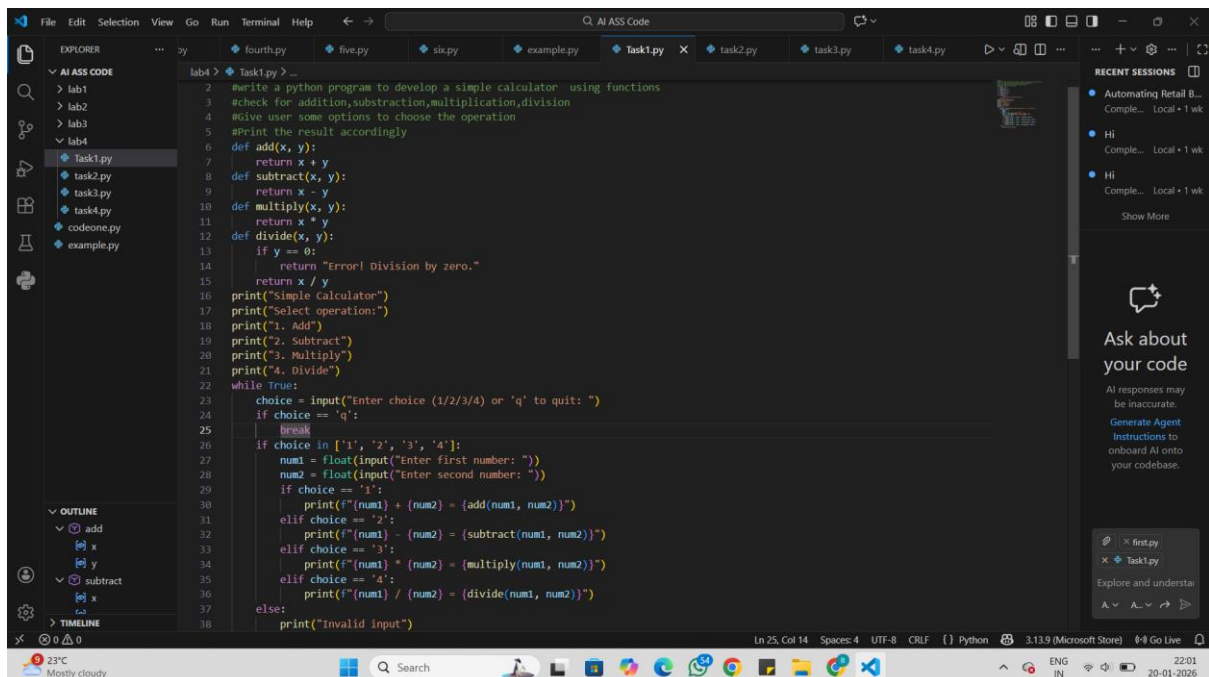
program by initially providing only the function name. Gradually enhance the prompt by

adding comments and usage examples.

Expected Output-1

- Comparison showing improvement in AI-generated calculator logic and structure.

Code and Output:



```
lab4 > Task1.py > ...
2 #write a python program to develop a simple calculator using functions
3 #check for addition, subtraction, multiplication, division
4 #give user some options to choose the operation
5 #print the result accordingly
6 def add(x, y):
7     return x + y
8 def subtract(x, y):
9     return x - y
10 def multiply(x, y):
11     return x * y
12 def divide(x, y):
13     if y == 0:
14         return "Error! Division by zero."
15     return x / y
16 print("Simple Calculator")
17 print("Select operation:")
18 print("1. Add")
19 print("2. Subtract")
20 print("3. Multiply")
21 print("4. Divide")
22 while True:
23     choice = input("Enter choice (1/2/3/4) or 'q' to quit: ")
24     if choice == 'q':
25         break
26     if choice in ['1', '2', '3', '4']:
27         num1 = float(input("Enter first number: "))
28         num2 = float(input("Enter second number: "))
29         if choice == '1':
30             print(f"{num1} + {num2} = {add(num1, num2)}")
31         elif choice == '2':
32             print(f"{num1} - {num2} = {subtract(num1, num2)}")
33         elif choice == '3':
34             print(f"{num1} * {num2} = {multiply(num1, num2)}")
35         elif choice == '4':
36             print(f"{num1} / {num2} = {divide(num1, num2)}")
37     else:
38         print("Invalid input")
```

The screenshot shows a Visual Studio Code editor with a Python file named `task1.py` open. The code is a simple calculator that takes user input for operations and numbers, and prints the result. The terminal at the bottom shows the execution of the program, with the user entering '2' for subtraction, '398449' for the first number, and '2359001' for the second number, resulting in '- 39448.0 = 2359001.0'.

```
19 print("2. Subtract")
20 print("3. Multiply")
21 print("4. Divide")
22 while True:
23     choice = input("Enter choice (1/2/3/4) or 'q' to quit: ")
24     if choice == 'q':
25         break
26     if choice in ['1', '2', '3', '4']:
27         num1 = float(input("Enter first number: "))
28         num2 = float(input("Enter second number: "))
29         if choice == '1':
30             print(f"{num1} + {num2} = {add(num1, num2)}")
31         elif choice == '2':
32             print(f"{num1} - {num2} = {subtract(num1, num2)}")
33         elif choice == '3':
34             print(f"{num1} * {num2} = {multiply(num1, num2)}")
35         elif choice == '4':
36             print(f"{num1} / {num2} = {divide(num1, num2)}")
37     else:
38         print("Invalid input")
39
```

Terminal Output:

```
PS C:\Users\tejas\College\AI ASS Code> & C:\Users\tejas\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/tejas/College/AI ASS Code/lab4/Task1.py"
Select operation:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice (1/2/3/4) or 'q' to quit: 2
Enter first number: 398449
Enter second number: 2359001
398449.0 - 39448.0 = 2359001.0
Enter choice (1/2/3/4) or 'q' to quit: 
```

Analysis:

Analysis: Progressive Prompting for Calculator Design

Progressive prompting means giving instructions step by step to the AI.

First, only the function name is given, so the output is basic.

Next, comments are added to explain what the calculator should do.

Then, usage examples are included to show how the program works.

Each step improves clarity and accuracy of the AI response.

This method helps in getting a complete and correct calculator program.

Task Description-2

- Refining Prompts for Sorting Logic: Start with a vague prompt for sorting student marks,

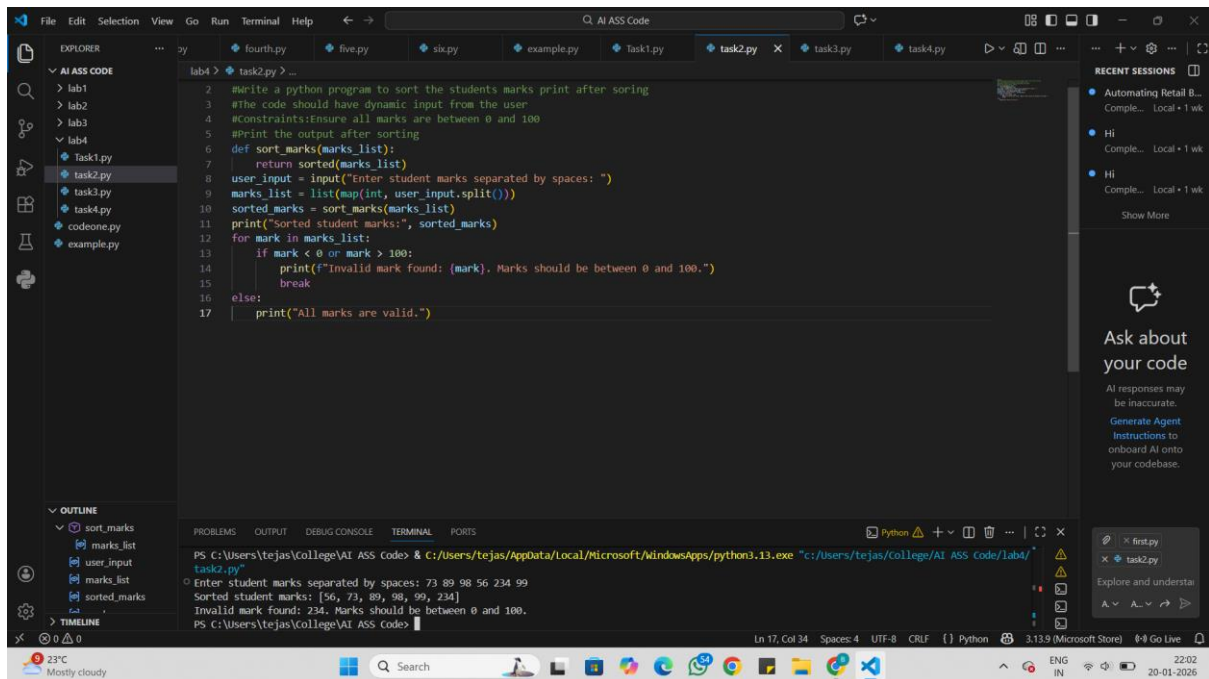
then refine it to clearly specify sorting order and constraints.

Expected Output-2

- AI-generated sorting function evolves from ambiguous logic to an accurate and efficient

implementation.

Code and Output:



```
File Edit Selection View Go Run Terminal Help
AI ASS CODE
lab1 > task2.py > ...
lab1
lab2
lab3
lab4
Task1.py
task2.py
task3.py
task4.py
codeone.py
example.py
OUTLINE
sort_marks
marks_list
user_input
marks_list
sorted_marks
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python 3.13.9 (Microsoft Store)
PS C:\Users\tejas\College\AI ASS Code> & C:\Users\tejas\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:\Users\tejas\College\AI ASS Code\lab4\task2.py"
Enter student marks separated by spaces: 73 89 98 56 234 99
Sorted student marks: [56, 73, 89, 98, 99, 234]
Invalid mark found: 234. Marks should be between 0 and 100.
PS C:\Users\tejas\College\AI ASS Code>
```

Analysis:

Refining Prompts for Sorting Logic

Initially, a vague prompt is given to sort student marks.

Due to less clarity, the AI may give an incomplete or unclear solution.

The prompt is then refined to specify sorting order (ascending or descending).

Additional constraints (such as valid marks or number of students) are added.

Clear instructions help the AI produce accurate and expected output.

Refining prompts reduces confusion and errors in the solution.

Task Description-3

- Few-Shot Prompting for Prime Number Validation: Provide multiple input-output

examples for a function that checks whether a number is prime. Observe how few-shot

prompting improves correctness.

Expected Output-3

- Improved prime-checking function with better edge-case handling.

Code and Output:

```

5 def is_prime(number):
6     #check for edge cases
7     if number <= 1:
8         return False
9     if number <= 3:
10        return True
11    if number % 2 == 0 or number % 3 == 0:
12        return False
13    i = 5
14    while i * i <= number:
15        if number % i == 0 or number % (i + 2) == 0:
16            return False
17        i += 6
18    return True
19 user_input = int(input("Enter a number: "))
20 if is_prime(user_input):
21     print(f"{user_input} is a Prime Number.")
22 else:
23     print(f"{user_input} is Not a Prime Number.")

```

Terminal Output:

```

PS C:\Users\tejas\College\AI ASS Code> & C:\Users\tejas\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:\Users\tejas\College\AI ASS Code\lab4\task3.py"
Enter a number: 2098
2098 is Not a Prime Number.
PS C:\Users\tejas\College\AI ASS Code>

```

Analysis:

Few-Shot Prompting for Prime Number Validation

Few-shot prompting means giving multiple input-output examples to the AI.

The examples show how prime and non-prime numbers behave.

This helps the AI understand the exact logic of prime number checking.

The AI produces more correct and reliable results.

Errors are reduced compared to a prompt with no examples.

Few-shot prompting improves accuracy and consistency of the function.

Task Description-4

- Prompt-Guided UI Design for Student Grading System: Create a user interface for a

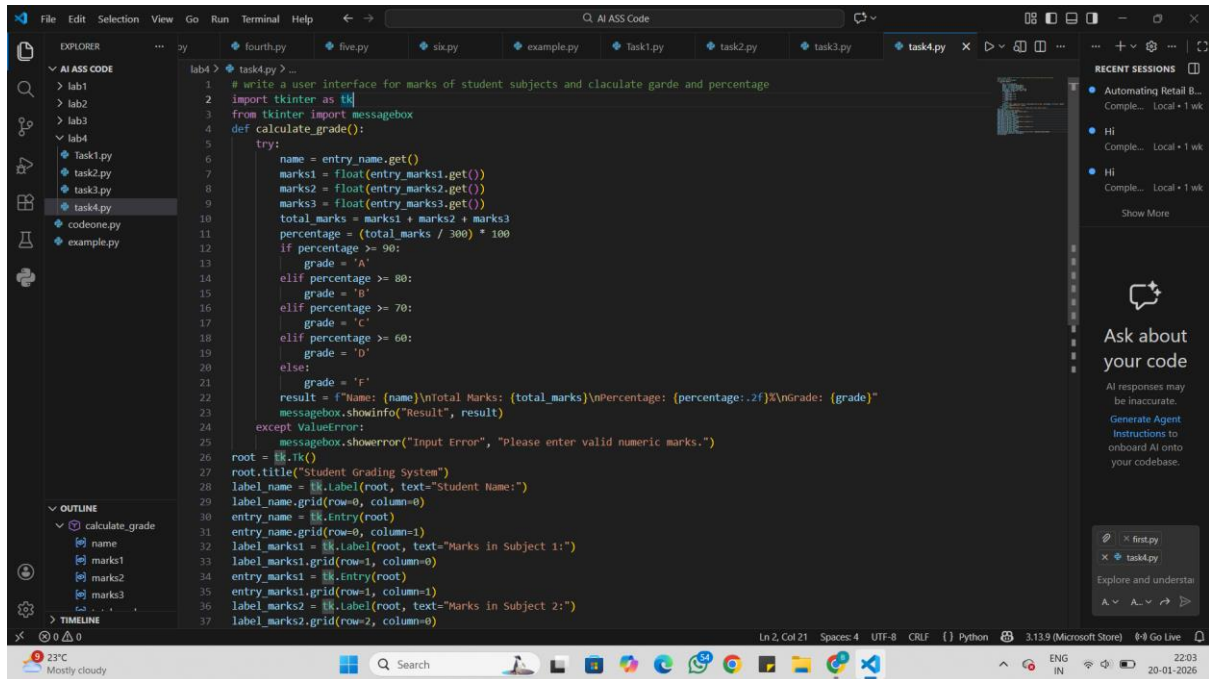
student grading system that calculates total marks, percentage, and grade based on user

input.

Expected Output-4

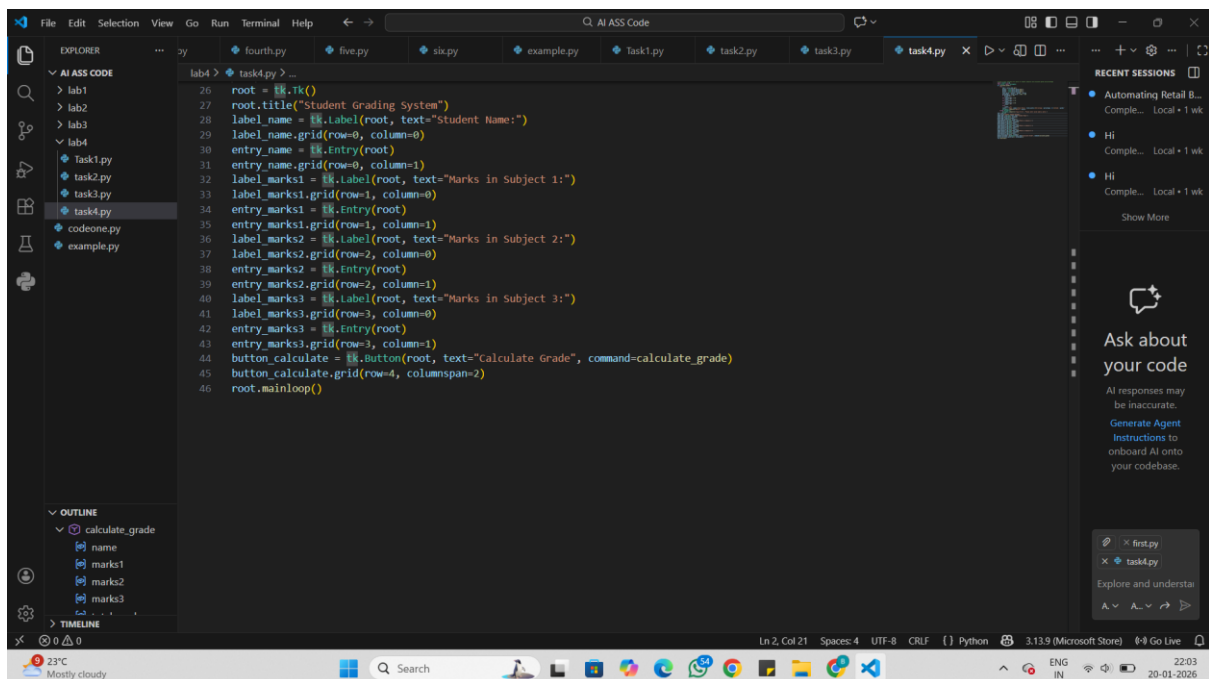
- Well-structured UI code with accurate calculations and clear output display.

Code and Output:



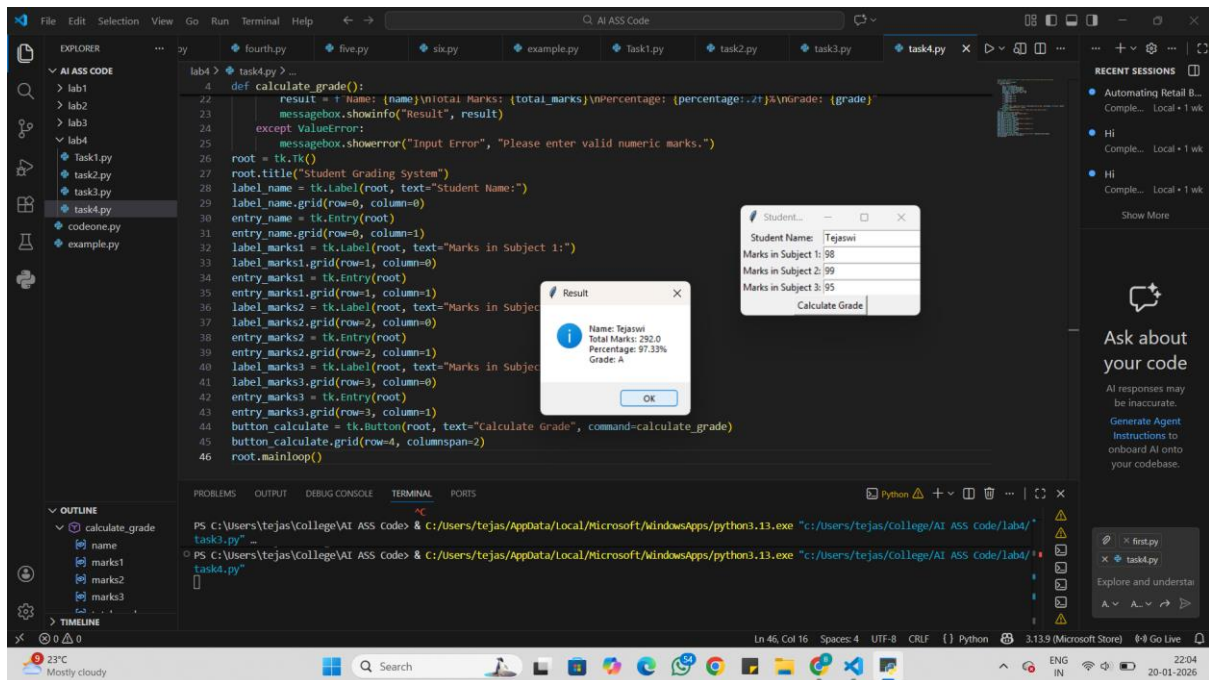
This screenshot shows the first part of the Python code in a VS Code editor. The code defines a function `calculate_grade()` that takes student marks and calculates a grade and percentage. The code is as follows:

```
1 # write a user interface for marks of student subjects and calculate grade and percentage
2 import tkinter as tk
3 from tkinter import messagebox
4 def calculate_grade():
5     try:
6         name = entry_name.get()
7         marks1 = float(entry_marks1.get())
8         marks2 = float(entry_marks2.get())
9         marks3 = float(entry_marks3.get())
10        total_marks = marks1 + marks2 + marks3
11        percentage = (total_marks / 300) * 100
12        if percentage >= 90:
13            grade = 'A'
14        elif percentage >= 80:
15            grade = 'B'
16        elif percentage >= 70:
17            grade = 'C'
18        elif percentage >= 60:
19            grade = 'D'
20        else:
21            grade = 'F'
22        result = f'Name: {name}\nTotal Marks: {total_marks}\nPercentage: {percentage:.2f}%\nGrade: {grade}'
23        messagebox.showinfo("Result", result)
24    except ValueError:
25        messagebox.showerror("Input Error", "Please enter valid numeric marks.")
26
27 root = Tk()
28 root.title("Student Grading System")
29 label_name = Tk.Label(root, text="Student Name:")
30 label_name.grid(row=0, column=0)
31 entry_name = Tk.Entry(root)
32 entry_name.grid(row=0, column=1)
33 label_marks1 = Tk.Label(root, text="Marks in Subject 1:")
34 label_marks1.grid(row=1, column=0)
35 entry_marks1 = Tk.Entry(root)
36 entry_marks1.grid(row=1, column=1)
37 label_marks2 = Tk.Label(root, text="Marks in Subject 2:")
38 label_marks2.grid(row=2, column=0)
```



This screenshot shows the second part of the Python code in a VS Code editor. The code continues from the previous part, defining the UI elements and the `mainloop()` function. The code is as follows:

```
26 root = Tk()
27 root.title("Student Grading System")
28 label_name = Tk.Label(root, text="Student Name:")
29 label_name.grid(row=0, column=0)
30 entry_name = Tk.Entry(root)
31 entry_name.grid(row=0, column=1)
32 label_marks1 = Tk.Label(root, text="Marks in Subject 1:")
33 label_marks1.grid(row=1, column=0)
34 entry_marks1 = Tk.Entry(root)
35 entry_marks1.grid(row=1, column=1)
36 label_marks2 = Tk.Label(root, text="Marks in Subject 2:")
37 label_marks2.grid(row=2, column=0)
38 entry_marks2 = Tk.Entry(root)
39 entry_marks2.grid(row=2, column=1)
40 label_marks3 = Tk.Label(root, text="Marks in Subject 3:")
41 label_marks3.grid(row=3, column=0)
42 entry_marks3 = Tk.Entry(root)
43 entry_marks3.grid(row=3, column=1)
44 button_calculate = Tk.Button(root, text="Calculate Grade", command=calculate_grade)
45 button_calculate.grid(row=4, columnspan=2)
46 root.mainloop()
```



Analysis:

Prompt-Guided UI Design for Student Grading System

Prompt-guided design gives clear instructions for UI creation.

The prompt specifies inputs like student marks.

The system calculates total marks and percentage automatically.

It assigns grades based on rules given in the prompt.

Clear prompts help design a user-friendly interface.

Prompt guidance reduces errors and confusion in UI behavior.

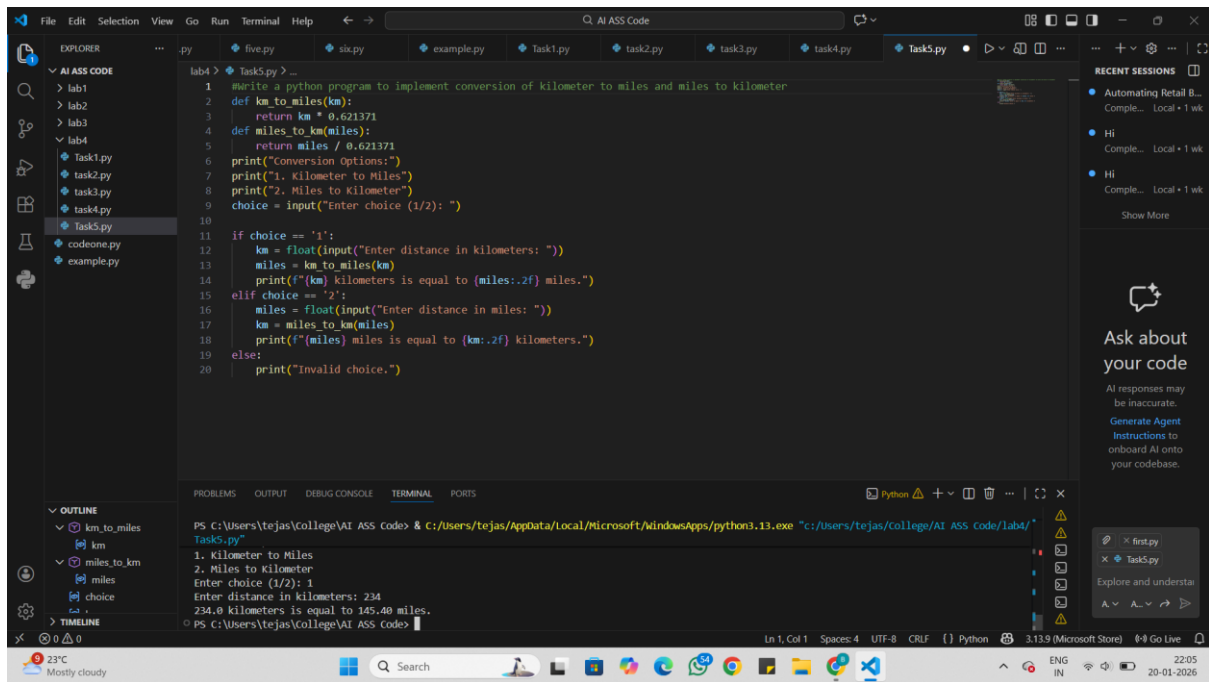
Task Description-5

- Analyzing Prompt Specificity in Unit Conversion Functions: Improving a Unit Conversion Function (Kilometers to Miles and Miles to Kilometers) Using Clear Instructions.

Expected Output-5

- Analysis of code quality and accuracy differences across multiple prompt variations.

Code and Output:



Analysis:

Prompt Specificity in Unit Conversion Functions

Prompt specificity means giving clear and detailed instructions.

Initially, a vague prompt may cause incorrect or incomplete conversion.

Clear instructions specify conversion type (km to miles, miles to km).

The prompt defines input and output format clearly.

Specific prompts improve accuracy and correctness of the function.

This reduces errors and misunderstanding in unit conversion.